MulitaMiner: especificação técnica e plano de desenvolvimento e avaliação

Beatriz Machado · Cristhian Kapelinski · Guilherme Klauberg · Universidade Federal do Pampa (UNIPAMPA) · Laboratório de IA Aplicada · Atividade 3

1. ESPECIFICAÇÃO DO PROBLEMA

Tarefa. Dado um relatório em PDF gerado por um scanner de vulnerabilidades, produzir um registro estruturado por achado, num esquema único com dez campos (nome, severidade, CVSS, host, porta, protocolo, descrição, solução, impacto, referências) mais a origem, de modo que o número de registros seja igual ao número de achados impressos no relatório. **Usuário.** O analista de segurança ou equipe de resposta a incidentes (CSIRT) que recebe relatórios de várias ferramentas e precisa consolidar, comparar e priorizar. **Entrada.** O PDF exatamente como o scanner o exporta, para OpenVAS, Nessus, Qualys e ZAP. **Saída.** Um arquivo JSON com um objeto por achado e a mesma tabela em XLSX e CSV; formatos de integração (SARIF, CSAF) ficam como extensão. **Cenário.** Um lote de relatórios de uma varredura periódica processado numa máquina da própria organização, sem que o conteúdo, que descreve hosts, portas e falhas da rede, seja enviado a terceiros; a execução em nuvem existe apenas como referência de comparação.

Dentro do escopo: os quatro scanners, relatórios em inglês, os dez campos, execução local e em nuvem, linha de comando. **Fora do escopo:** PDF digitalizado (OCR), outros idiomas, correlação entre varreduras ao longo do tempo, interface gráfica, e scanners além dos quatro (a arquitetura prevê, mas não avaliaremos).

2. DADOS

Fonte. Varreduras de aplicações intencionalmente vulneráveis (OWASP Juice Shop, bWAPP, WackoPicko, versões antigas de WordPress e Drupal) com os quatro scanners. Cada varredura gera um par: o PDF, que é a entrada do sistema, e a exportação de máquina do mesmo scanner (XLSX, CSV ou XML), que é o gabarito. **Quantidade e formato.** 136 relatórios, 6.841 achados pareados; 6.187 para treino e 654 para validação, mais 8 relatórios reservados para teste (1.028 achados), nunca usados no ajuste. Cada exemplo é um bloco de texto do PDF e o registro JSON correspondente. **Rótulos.** Derivados da exportação, sem anotação manual: o pareamento usa host mais identificador do achado (nome do NVT no OpenVAS, QID no Qualys, plugin id no Nessus, nome do alerta no ZAP). **Preparação.** Extração do texto do PDF, remoção de cabeçalhos, rodapés e quebras de página, segmentação em um bloco por achado, pareamento, normalização de severidade e CVSS, e exclusão dos trechos do gabarito que o PDF não imprime, para o modelo não aprender a inventar.

Problema previsto	Por que importa	Tratamento
Desbalanceamento	Cerca de 80% dos exemplos são OpenVAS	Métricas reportadas por scanner, além do agregado
Fidelidade do gabarito	A exportação traz texto ausente do PDF, ou omite campo que o PDF mostra	Auditoria campo a campo antes de usar como métrica; trechos ausentes saem do rótulo
Vazamento treino/teste	Relatórios da mesma aplicação repetem achados	Separação por aplicação, não por arquivo
Privacidade e segurança	O relatório é o mapa da superfície de ataque	Dados só de laboratório; o sistema proposto roda sem enviar nada à nuvem

3. ABORDAGEM DE IA

Técnica investigada. Um modelo de linguagem generativo decodificando sob restrição de esquema JSON: recebe um lote de blocos numerados e devolve exatamente um objeto por id. Escolhemos gerar em vez de rotular sequência porque a saída não é um recorte literal do texto: solução e impacto precisam ser reescritos, e a severidade precisa ser mapeada de vocabulários distintos para um conjunto único. O id fixo por bloco permite detectar omissão e invenção por construção. **Baseline.** Um extrator por regras (expressões regulares sobre o cabeçalho e os títulos de seção de cada bloco), o piso que qualquer modelo precisa superar nos campos narrativos. **Alternativas comparadas.** (a) *Teto*: modelo de nuvem few-shot (DeepSeek V4.1 Flash via API); (b) *local sem treino*: modelo pequeno executado numa GPU da organização, também few-shot, Qwen2.5 1,5B Instruct como primário e um segundo modelo de controle; (c) *sistema proposto*: o mesmo modelo local ajustado com QLoRA em 4 bits [Dettmers et al. 2023] sobre os pares de treino. Não usaremos RAG: todo o contexto necessário está no próprio bloco. **Recursos.** Uma GPU com 16 GB de VRAM (RTX 5080) para ajuste fino, estimado em uma a duas horas por modelo de 1,5B, e para inferência local; CPU basta para baseline e segmentação; crédito de API da ordem de US$ 1 por rodada completa no teto.

4. ARQUITETURA

Um pipeline fixo com o modelo como peça trocável, implementado como pacote Python de linha de comando:

- **Leitura:** pypdfium2 extrai o texto do PDF, sem OCR.
- **Limpeza:** cabeçalhos repetidos, rodapés e marcas de continuação de página são removidos antes do modelo.
- **Segmentação:** uma configuração JSON por scanner diz qual linha abre um achado; sai um bloco por achado, com id fixo e host e porta lidos do cabeçalho. A contagem fica definida aqui, antes de qualquer chamada ao modelo.
- **Empacotamento:** blocos inteiros são agrupados em lotes por orçamento de tokens; nenhum bloco é cortado.
- **Extração:** chamada ao modelo por uma API compatível com OpenAI, na nuvem ou num servidor local (Ollama ou vLLM), com o esquema JSON e um prompt por scanner; a resposta é validada com Pydantic e um lote recusado volta em pacotes menores, até três tentativas.
- **Consolidação:** normalização de severidade e CVSS e fusão de registros duplicados.
- **Exportação:** JSON, XLSX e CSV; outros formatos como extensão.

A arquitetura é realista para o semestre: tudo roda numa máquina com uma GPU, baseline e segmentação não dependem do modelo, e trocar de modelo é trocar um argumento. Um scanner novo entra com um JSON e um prompt, sem código.

5. PROTÓTIPO MÍNIMO

MVP. O caminho completo para um scanner (OpenVAS, o de maior volume), com um modelo local few-shot, produzindo JSON e XLSX, acompanhado do baseline por regras e de um script que mede cobertura e acerto exato contra a exportação do scanner. **Essencial:** leitura, segmentação, chamada com esquema e validação, consolidação, exportação JSON e XLSX, métricas de cobertura e de campos curtos. **Posterior:** os outros três scanners, o ajuste fino, as métricas de texto, SARIF e CSAF, e uma interface web.

6. PLANO EXPERIMENTAL

Perguntas. Q1 A extração com LLM supera o baseline por regras nos campos narrativos? **Q2** Quanto se perde ao trocar o modelo de nuvem por um local sem treino? **Q3** O ajuste fino recupera essa diferença? **Q4** Qual configuração tem a melhor relação entre qualidade e custo (tempo, dinheiro, VRAM)? **Q5** O comportamento se mantém num scanner ausente do treino?

Dimensão	Métrica	Pergunta que responde
Cobertura	Recall de blocos (achados que viraram registro) e achados inventados	Q1 a Q3, Q5
Campos curtos	Acerto exato em severidade, porta, protocolo, CVSS e id do teste; macro-F1 da severidade	Q1 a Q3
Campos narrativos	Similaridade lexical (ROUGE-L) e semântica (BERTScore) em descrição e solução; F1 de conjunto em referências	Q1 a Q3
Robustez e custo	% de JSON válido, tempo por relatório, VRAM, tokens e custo de API	Q4

Protocolo. Os 8 relatórios de teste, com 5 rodadas por configuração, reportando média e desvio, por scanner e agregado. Configurações: baseline por regras, nuvem few-shot, local few-shot (primário e controle) e local ajustado, todas sobre o mesmo pipeline e os mesmos prompts. Um corte adicional treina sem um scanner e testa nele (Q5).

7. CRITÉRIOS DE SUCESSO

Critério	Valor considerado satisfatório
Cobertura do sistema proposto	Recall de blocos igual ou superior a 95% e achados inventados em no máximo 1% do total, em cada scanner
Distância para o teto	No máximo 5 pontos percentuais abaixo do modelo de nuvem em cobertura
Campos curtos	Acerto exato de severidade igual ou superior a 95% em cada scanner
Campos narrativos	ROUGE-L médio da descrição igual ou superior a 0,80 e acima do baseline por regras em todos os campos narrativos

Custo	Tempo por relatório igual ou menor que o da nuvem, custo de API zero, JSON válido em pelo menos 99% das chamadas

8. DESENVOLVIMENTO

Semanas	Etapa	Entrega
1 a 2	Dados	Pares PDF e exportação coletados, pareamento por host e id, separação treino, validação e teste congelada
3	Baseline	Extrator por regras e script de avaliação de cobertura e campos curtos
4 a 5	Pipeline (MVP)	Leitura, segmentação, chamada com esquema, consolidação e exportação para o OpenVAS; depois os outros scanners
6	Teto	Rodadas com o modelo de nuvem; métricas de texto implementadas
7	Seleção	Modelos locais few-shot (primário e controle) servidos localmente
8 a 9	Ajuste fino	Treino QLoRA e validação; escolha de hiperparâmetros
10 a 12	Experimentos	5 rodadas por configuração, corte de scanner não visto, análise de erros por tipo
13 a 14	Fechamento	Resultados, documentação, repositório e apresentação final

9. RISCOS E PLANO B

Risco	Alternativa
Gabarito incompleto ou divergente do PDF em algum campo	Auditar o campo; se não houver gabarito confiável, excluí-lo das métricas e reportar só os demais
Modelo de 1,5B insuficiente mesmo ajustado	Subir para 3B ou 7B em 4 bits, aumentar épocas, ou restringir o escopo aos campos em que ele acerta
GPU indisponível ou com pouca VRAM	Inferência em CPU para o conjunto de teste (lento, mas viável) e treino pontual em GPU alugada
Segmentação falha num scanner	Corrigir a configuração; se persistir, retirar o scanner do escopo e documentar
Janela de contexto do modelo local	Lotes menores e limite de tokens por bloco; blocos grandes vão sozinhos
Tempo do semestre	Priorizar OpenVAS e a comparação nuvem versus local; ajuste fino em um scanner só, se preciso

10. USO DE IA

ChatGPT e Claude foram usados para discutir e criticar a proposta, e o Claude Code, sobre o repositório, para rascunhar este documento, escrever scripts de contagem (funil de pareamento, exemplos por scanner) e trechos de código do pipeline. **Aproveitadas:** separar a avaliação em cobertura e conteúdo; tratar local ou nuvem como variável experimental sobre o mesmo pipeline; fixar o id do bloco para detectar omissão e invenção. **Modificadas:** a faixa de modelos locais sugerida (4B a 21B) foi reduzida a 1,5B por causa da VRAM e do tempo de treino disponíveis. **Descartadas:** usar um LLM como juiz da saída, dispensável com gabarito de máquina, e extrair tudo por regras, que não cobre os campos narrativos. **Verificação:** todo número deste documento vem de scripts executados sobre os artefatos do projeto, o código sugerido é coberto por testes automatizados, e as sugestões sobre dados foram conferidas contra as exportações dos scanners.

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais, art. 46.

DETTMERS, T.; PAGNONI, A.; HOLTZMAN, A.; ZETTLEMOYER, L. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS, 2023.

MACHADO, B.; LAUTERT, D.; KAPELINSKI, C.; KREUTZ, D. Structured Extraction of Vulnerabilities in OpenVAS and Tenable WAS Reports Using LLMs. Anais da XXII ERRC, SBC, 2025.